Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Distributed Computing

Practical Federated RL: Implementation and Interactive Scenario Design with Flower.ai

Bachelor's Thesis

Joel Maag

maagjo@ethz.ch

Distributed Computing Group
Computer Engineering and Networks Laboratory
ETH Zürich

Supervisors:
Flint Xiaofeng Fan
Prof. Dr. Roger Wattenhofer

April 5, 2026

Acknowledgements

I would like to thank my supervisor Flint Xiaofeng Fan for his guidance and feedback throughout this project. His suggestions helped sharpen both the technical direction and the presentation of this work. I am also grateful to Prof. Dr. Roger Wattenhofer and the DISCO group at ETH Zürich for providing the research environment in which this thesis was carried out.

Abstract

Federated reinforcement learning (FedRL) enables multiple agents to collaboratively train a shared policy without exchanging raw environment data, making it attractive for privacy-sensitive and distributed deployments. However, the absence of a standardized evaluation framework has made it difficult to compare algorithms across the literature: researchers implement their own training loops, environments, and baselines, yielding results that are rarely directly comparable.

This thesis presents an open benchmarking framework for federated reinforcement learning built directly on Flower.ai, a production federated learning framework. Our key insight is that building on real FL infrastructure rather than a custom simulation loop makes results directly transferable to practical deployments and reusable by anyone already working with Flower.ai. The framework ships with validated implementations of five established strategies (Independent Learning, Centralized Training, GPOMDP, SVRPG, and FedPG-BR), a plugin interface that allows a new strategy to be evaluated by implementing a single class, standardised experiment suites covering Byzantine robustness and multi-agent scalability, and a web-based dashboard for interactive scenario design.

We validate the framework by running all included strategies on CartPole-v1, LunarLander-v3, and the PettingZoo Pursuit environment. The results reproduce the expected relative ordering of methods from the original literature and confirm that the Byzantine filter implementations behave correctly under 30% corruption. The experiments demonstrate that the framework provides a consistent, reproducible substrate: any new strategy evaluated here is compared against the same baselines, the same environments, and the same adversarial conditions, making results directly comparable across papers. We argue that FedRL research requires shared evaluation infrastructure analogous to OpenAI Gym for single-agent RL, and we release the framework as a contribution to the Flower.ai baselines repository.

Contents

Chapter 1

Introduction

Reinforcement learning (RL) has achieved remarkable results in sequential decision-making tasks, from game playing to robotic control. In many real-world settings, however, the agents that must learn are not centralized; they are distributed across separate machines, each interacting with its own local environment. A traffic intersection cannot share raw sensor feeds with a central server due to bandwidth and privacy constraints. A fleet of robots operating in different locations cannot pool raw trajectory data. A hospital system training a treatment policy cannot share patient records. In all these cases, the agents must learn *together* without sharing raw data.

Federated reinforcement learning (FedRL) addresses this by having each agent train locally and share only model updates (typically policy parameters or gradients) with a central coordinator. The coordinator aggregates these updates and returns an improved global policy. This preserves data locality while enabling collaborative learning, combining the scalability of distributed systems with the privacy guarantees of federated learning [1].

1.1 Motivation

Despite growing interest in FedRL, the field lacks a shared experimental foundation. Each paper in the literature introduces its own training loop, its own environment setup, its own baseline implementations, and its own metric definitions. As a result, claims are difficult to verify and results across papers are rarely directly comparable. A new aggregation algorithm may appear superior simply because it was compared against a poorly tuned baseline, or evaluated on a different environment configuration.

This problem is well understood in single-agent RL: the introduction of OpenAI Gym [2] established a common interface for environments, enabling fair comparison across algorithms and accelerating research progress significantly. No equivalent standard exists for FedRL today.

A further challenge is the cost of entry. Building a correct federated RL

setup from scratch requires integrating a federated communication framework, implementing policy gradient estimation, handling gradient serialization across processes, and simulating adversarial clients for robustness evaluation. This infrastructure overhead discourages replication and diverts research effort away from algorithmic contributions.

1.2 Contributions

This thesis addresses these gaps with the following contributions:

1. **A federated RL benchmarking framework built on Flower.ai** that provides a unified substrate for evaluating FedRL strategies on real FL infrastructure. The framework includes a plugin interface allowing researchers to contribute a new strategy by implementing a single class, without modifying any infrastructure code.
2. **Validated baseline implementations** of five algorithms: Independent Learning, Centralized Training, FedPG-BR, GPOMDP, and SVRPG, all evaluated under identical conditions using the same Flower.ai infrastructure.
3. **Standardised experiment suites** covering Byzantine robustness (up to 30% malicious clients) and multi-agent scalability, together with a web-based dashboard for interactive scenario design and real-time monitoring.
4. **An empirical evaluation** across cooperative control tasks including CartPole, LunarLander, and the PettingZoo Pursuit environment, providing a reproducible baseline for future algorithmic comparisons.

The framework is developed as an open-source contribution intended for submission to the Flower.ai baselines repository, with the goal of providing a shared foundation for federated RL research in the same way that established benchmarks have standardized other subfields.

1.3 Thesis Outline

The remainder of this thesis is structured as follows. Chapter 2 reviews the necessary background in reinforcement learning, policy gradient methods, federated learning, and Byzantine robustness. Chapter 3 presents the algorithms implemented as baselines. Chapter 4 describes the system architecture and design decisions behind the benchmarking framework. Chapter 5 motivates and describes the benchmarking framework in detail, including the plugin interface and

experiment suite design. Chapter 6 presents the experimental results and analysis. Chapter 7 concludes with a summary of findings, limitations, and directions for future work.

Chapter 2

Background

This chapter reviews the foundational concepts underlying this thesis. Section 2.1 introduces reinforcement learning and policy gradient methods. Section 2.2 covers federated learning and aggregation. Section 2.3 addresses Byzantine robustness. Section 2.4 surveys related work in federated reinforcement learning.

2.1 Reinforcement Learning

Reinforcement learning concerns an agent interacting with an environment over discrete time steps. At each step t, the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi_\theta(a \mid s)$, and receives a scalar reward $r_t \in \mathbb{R}$. The environment transitions to a new state s_{t+1} according to a transition distribution $P(s_{t+1} \mid s_t, a_t)$. This interaction is formally modelled as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\gamma \in [0, 1)$ is a discount factor [3].

The agent's objective is to find a policy π_θ that maximises the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right], \tag{2.1}$$

where $\tau = (s_0, a_0, r_0, \ldots, s_T)$ denotes a trajectory sampled under π_θ.

2.1.1 Policy Gradient Methods

Policy gradient methods optimise $J(\theta)$ directly by computing the gradient with respect to the policy parameters θ. The policy gradient theorem [3] gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t \right], \tag{2.2}$$

where $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$ is the return from step t. The REINFORCE algorithm [4] estimates this gradient using Monte Carlo rollouts:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \cdot G_t^{(i)}, \tag{2.3}$$

where N trajectories are collected per update. While unbiased, this estimator has high variance, particularly in environments with long horizons.

2.1.2 Variance Reduction: SVRPG

Stochastic Variance-Reduced Policy Gradient (SVRPG) [5] adapts variance reduction techniques from stochastic optimisation to the policy gradient setting. A snapshot gradient is computed periodically at a reference parameter $\tilde{\theta}$, and individual updates correct for the difference between the current and snapshot gradient. This reduces gradient variance without introducing bias, leading to faster convergence in practice.

2.2 Federated Learning

Federated learning (FL) enables a set of K clients to jointly train a shared model without exchanging raw data [1]. Each client k holds a local dataset $\mathcal{D}_k$ and computes a local update (typically a gradient or model delta) which is sent to a central server. The server aggregates these updates and broadcasts the improved global model back to clients.

The canonical algorithm, FedAvg [6], aggregates client updates as a weighted average:

$$\theta_{\text{global}} \leftarrow \sum_{k=1}^{K} \frac{n_k}{n} \theta_k, \tag{2.4}$$

where n_k is the number of data points at client k and $n = \sum_k n_k$. Communication occurs in rounds: each round consists of a server broadcast, local client updates, and an aggregation step.

In the reinforcement learning setting, clients do not hold static datasets but instead collect trajectories by interacting with their local environments. The "data" generated at each client depends on the current policy, introducing a non-stationarity that distinguishes FedRL from standard FL.

2.3 Byzantine Robustness

A Byzantine client is one that may send arbitrary, potentially adversarial updates to the server [7]. Byzantine failures can arise from hardware faults, software bugs,

or deliberate attacks. Under FedAvg, even a single Byzantine client can corrupt the global model by sending a sufficiently large gradient.

Robust aggregation rules replace the mean with a statistic that is resistant to outliers. Two common approaches are:

- **Krum** [8]: selects the client update with the smallest sum of distances to its $K - f - 2$ nearest neighbours, where f is the assumed number of Byzantine clients.
- **Coordinate-wise median**: replaces the mean with the coordinate-wise median across client updates, which is robust to up to $\lfloor K/2 \rfloor - 1$ Byzantine clients under mild assumptions.

FedPG-BR [9] applies a filtering mechanism specifically adapted to the policy gradient setting, discarding updates that deviate significantly from the estimated gradient direction before aggregation.

2.4 Related Work

Several works have studied policy gradient methods in federated and multi-agent settings. Table 2.1 summarises the key properties of each and positions them against this thesis.

Table 2.1: Comparison of related FedRL works. **Byz. robust**: tolerates Byzantine clients. **Variance red.**: uses variance reduction in gradient estimation. **Multi-env**: evaluated on more than one environment. **Real FL stack**: built on a production federated learning framework (not a custom simulation loop). **Shared codebase**: implementation is public and reusable. **Defines eval. problem**: provides a formal definition of the evaluation setup.

Work	Byz. robust	Variance red.	Multi-env	Real FL stack	Shared codebase	Defines eval. problem
FedPG [9]	×	×	×	×	×	×
GPOMDP [10]	×	×	×	×	×	×
SVRPG [5]	×	✓	×	×	×	×
FedPG-BR [9]	✓	✓	×	×	×	×
This work	✓	✓	✓	✓	✓	✓

FedPG [9] extends REINFORCE to the federated setting by averaging client gradients at the server, but provides no robustness guarantees and is evaluated on a single environment using a custom simulation loop. **GPOMDP** [10] addresses global optimisation of a shared MDP across heterogeneous clients but shares the same limitations. **SVRPG** [5] improves sample efficiency through variance reduction yet remains fragile under adversarial clients and relies on a standalone implementation that cannot be reused. **FedPG-BR** [9] adds Byzantine robustness via gradient filtering, but like all prior work, implements federation as a custom loop disconnected from any real FL infrastructure.

We differ from all prior work in one key structural way: our framework is built directly on Flower.ai [11], a production federated learning framework with real client–server communication, meaning experiments run on actual FL infrastructure rather than a custom simulation abstraction. This closes the gap between research evaluation and real deployment, and makes the benchmark directly reusable by practitioners already using Flower.ai.

Chapter 3

Baseline Implementations

A benchmarking framework is only as useful as its baselines. This chapter documents the five strategies implemented in the benchmark, the papers they are taken from, and the design choices made during implementation. All algorithms are reproduced from their respective original publications; no algorithmic novelty is claimed here. The goal is to provide faithful, reproducible implementations that serve as a fair reference point for future work.

Table 3.1 summarises the properties of each strategy.

Table 3.1: Summary of implemented strategies and their source papers.

Strategy	**Source**	**Federated**	**Variance Reduction**	**Byzantine Robust**
Independent	(baseline)	No	No	No
Centralized	(baseline)	No	No	No
GPOMDP	[10]	Yes	No	No
SVRPG	[5]	Yes	Yes	No
FedPG-BR	[9]	Yes	Yes	Yes

3.1 Design Baselines

Two non-federated baselines are included to bracket the expected performance range. These are not taken from the literature but are standard evaluation anchors in the FedRL setting.

Independent Learning simulates the case in which agents never share information. Each round, only a single worker's gradient is used to update the policy; all other workers' gradients are discarded. This represents training a single agent in isolation and serves as a lower bound: a federated method that cannot outperform it provides no benefit from federation.

Centralized Training simulates the oracle upper bound: a single agent with access to $K \times B$ trajectories per round, where K is the number of clients and B is the per-client batch size. This is achieved by running a single worker

with a proportionally scaled batch size. Performance at or above this bound is not expected from a federated method; the gap between a federated method and Centralized quantifies the cost of distributing data across clients.

3.2 GPOMDP

GPOMDP [10] is the simplest federated policy gradient method and the starting point for the other algorithms in this chapter. We implement it as described in the original paper. All K client gradients are averaged and a single gradient step is taken:

$$\mu_t = \frac{1}{K}\sum_{k=1}^{K}\hat{g}_k, \qquad \theta_{t+1} \leftarrow \theta_t + \eta\mu_t. \tag{3.1}$$

There is no Byzantine filtering and no variance reduction. We include GPOMDP as the vanilla federation baseline to isolate the contribution of each extension: comparing GPOMDP against SVRPG reveals the benefit of variance reduction, and comparing GPOMDP against FedPG-BR reveals the combined benefit of filtering and variance reduction.

3.3 SVRPG

SVRPG [5] extends GPOMDP with SCSG variance reduction at the server. We implement the algorithm as described in the original paper. After clients return gradients and the mean μ_t is formed, the server runs $N_t \sim \text{Geom}(p)$ additional corrected update steps, with $p = B_{\text{mini}}/(B + B_{\text{mini}})$. Each step computes:

$$v_t^n = \frac{1}{B_{\text{mini}}}\sum_{i=1}^{B_{\text{mini}}}\left[\hat{g}(\theta_t^n;\xi_i) - \hat{g}(\theta_t^0;\xi_i)\cdot\rho_i\right] + \mu_t, \tag{3.2}$$

where ρ_i is an importance weight correcting for the policy shift between θ_t^0 and θ_t^n. One implementation detail worth noting: we add an early-stopping condition that terminates the update loop if the mean importance ratio leaves $[0.995, 1.005]$, preventing updates when the snapshot policy has drifted too far for the correction to be reliable. This guard is consistent with the spirit of the original paper but is not explicitly stated there.

3.4 FedPG-BR

FedPG-BR [9] is the main algorithm evaluated in this benchmark. We implement it directly from the original paper, which presents it as two sub-procedures: FedPG-Aggregate (Byzantine filtering) and FedPG-Update (SCSG server update).

3.4.1 Byzantine Filtering (FedPG-Aggregate)

Given K client gradients, the filter computes a data-dependent threshold:

$$T_\mu = 2\sigma\sqrt{\frac{V}{B}}, \qquad V = 2\log\frac{2K}{\delta}, \tag{3.3}$$

where σ bounds gradient variance, B is the batch size, and δ is a confidence parameter. A candidate set S is formed by retaining gradients that have more than $K/2$ neighbours within distance T_μ. The momentum estimator μ^{mom} is the gradient in S closest to the mean of S. Any gradient farther than T_μ from μ^{mom} is rejected. If the retained set is smaller than $(1-\alpha)K$, the threshold relaxes to 2σ and the procedure repeats. The output is the mean over accepted gradients.

3.4.2 Server Update (FedPG-Update)

The server update reuses the SVRPG procedure from Section 3.3, with the filtered aggregate μ_t substituted as the snapshot gradient. No changes to this component were required.

3.4.3 Implementation Notes

The filtering step has $O(K^2)$ complexity in the number of clients, which is acceptable for the benchmark's default scale of 10 clients but would require attention at larger scales. The parameters σ and δ are per-environment hyperparameters defined in `config.py`; their values were taken from the ablation tables in the original paper.

Chapter 4

System Design

This chapter describes the architecture of the benchmarking framework. Section 4.1 gives a high-level overview. Section 4.2 describes the strategy plugin interface. Section 4.3 covers environment integration. Section 4.4 describes the configuration system. Section 4.5 covers metrics logging and the web dashboard.

4.1 Architecture Overview

The framework is built on top of Flower.ai [11], a framework for federated learning that provides a gRPC-based communication layer, a client management system, and a strategy abstraction for aggregation logic. The system follows the standard Flower server/client split: a `ServerApp` hosts the global policy and aggregation logic, while multiple `ClientApp` instances run independently, each interacting with their own local environment instance.

Training proceeds in communication rounds. At the start of each round, the server serialises the current global policy parameters and broadcasts them to all sampled clients via `configure_fit`. Each client deserialises the parameters, loads them into its local policy network, collects a batch of trajectories by interacting with its environment, computes a policy gradient estimate, and returns the gradient to the server. The server receives all client gradients in `aggregate_fit`, passes them through the selected aggregation strategy, updates the global policy, and begins the next round.

Evaluation runs every ten rounds: the server samples up to three clients, has each run ten evaluation episodes with the current global policy, and logs the average return. Additionally, the server itself evaluates the global policy directly at every round to provide a stable reward signal for monitoring.

4.2 Strategy Plugin Interface

The central design goal of the framework is that a researcher should be able to evaluate a new aggregation algorithm by writing a single Python class, without modifying any infrastructure code. This is achieved through a strategy registry.

Every strategy is a subclass of `AggregationStrategy` and must implement two methods:

- `aggregate(gradients, batch_size, **kwargs)`: receives a list of gradient tensors from K clients and returns a tuple of `(aggregated_gradient, good_agent_indices)`.
- `server_update(policy, optimizer, theta_t_0, mu_t, config, ...)`: applies the aggregated gradient to update the global policy and returns the number of update steps taken.

A strategy is registered by decorating the class with `@register_strategy("name")`. The decorator adds the class to a global registry keyed by name. At runtime, the server resolves the strategy from the registry using the `method` field in the run configuration. This means strategies are fully decoupled from the server: adding a new strategy requires no changes to `server_app.py` or any other file.

As a minimal example, a coordinate-wise median strategy can be implemented as:

```
@register_strategy("coordinate-median")
class CoordinateMedian(AggregationStrategy):
    description = "Coordinate-wise median aggregation"

    def aggregate(self, gradients, batch_size, **kwargs):
        stacked = torch.stack(gradients)
        median = torch.median(stacked, dim=0).values
        return median, list(range(len(gradients)))

    def server_update(self, policy, optimizer,
                      theta_t_0, mu_t, config):
        apply_gradient(policy, optimizer, mu_t)
        return 1
```

This class, placed in any file that is imported at startup, is sufficient to make the strategy available under the name `"coordinate-median"` in all experiment configurations. A step-by-step guide with the full template is given in Appendix A.

4.3 Environment Integration

Clients interact with environments through the standard Gymnasium interface [2]. The framework ships with pre-tuned hyperparameter configurations for CartPole-v1, MountainCar-v0, Acrobot-v1, and LunarLander-v3. Continuous-action environments (HalfCheetah-v2/v5) are configured but require MuJoCo and are not evaluated in this thesis. Each configuration specifies the discount factor γ, policy network architecture, learning rate, trajectory batch sizes, and the Byzantine filter parameters σ and δ.

Multi-agent environments from PettingZoo [12] are supported through a compatibility wrapper, `PettingZooSingleAgentWrapper`, which adapts a PettingZoo parallel environment to the Gymnasium interface. The wrapper assigns one agent index to the controlled agent; all other agents take uniformly random actions. This makes PettingZoo environments drop-in compatible with the existing training loop without modification. Two environments are registered by default: `Pursuit-v4` (cooperative pursuit, discrete actions) and `SimpleSpread-v3` (cooperative navigation, continuous actions). This thesis evaluates `Pursuit-v4`; `SimpleSpread-v3` is included as a registered configuration but not evaluated.

Environment and observation space dimensions are queried automatically at startup via `get_env_info`, which instantiates a temporary environment, inspects its observation and action spaces, and closes it. This eliminates the need to hard-code state or action dimensions in configuration files.

4.4 Configuration System

Experiments are configured through TOML files passed to the Flower run command. The run configuration specifies all experiment parameters: the environment name, number of communication rounds, number of clients, number of Byzantine clients, aggregation method, and hyperparameter overrides. This design separates experiment specification from code, enabling reproducible experiments that can be re-run by sharing a single configuration file.

Hyperparameters can be overridden at the run level; any parameter not specified falls back to the per-environment defaults defined in `config.py`. A global random seed (default 42) can also be set at the run level and is propagated to NumPy and PyTorch to ensure reproducibility across runs.

4.5 Metrics Logging and Dashboard

The server logs training metrics to TensorBoard [13] at every round. Logged quantities include the average client return, the number of good agents accepted

by the Byzantine filter, the number of SCSG update steps, and the L2 norm of the global policy parameters.

A web dashboard built with Flask and Socket.IO runs as a background thread within the server process. It receives per-round metrics via an HTTP POST to a local endpoint and displays live convergence curves, per-client status, and round-level statistics. When running inside Docker, the dashboard is exposed on port 8050 and can be accessed at `http://localhost:8050/experiment`. The dashboard is optional: if the Flask dependency is not installed, training proceeds without it.

Figure 4.1 shows the dashboard during a FedPG-BR run on CartPole-v1 with 3 Byzantine clients (30% corruption) under the FedPG Attack. The top panel displays the configuration, live reward curve, and per-client reward band. The middle panels show communication statistics (active vs. skipped workers per round) and the global policy parameter L2 norm. The worker grid (Figure 4.3) identifies Byzantine workers (W8–W10) with a warning indicator. Figure 4.4 shows the training log with per-round statistics.

Figure 4.1: Dashboard overview: configuration panel, live learning curve, and per-client reward band. FedPG-BR on CartPole-v1 with 3 Byzantine clients under the FedPG Attack.

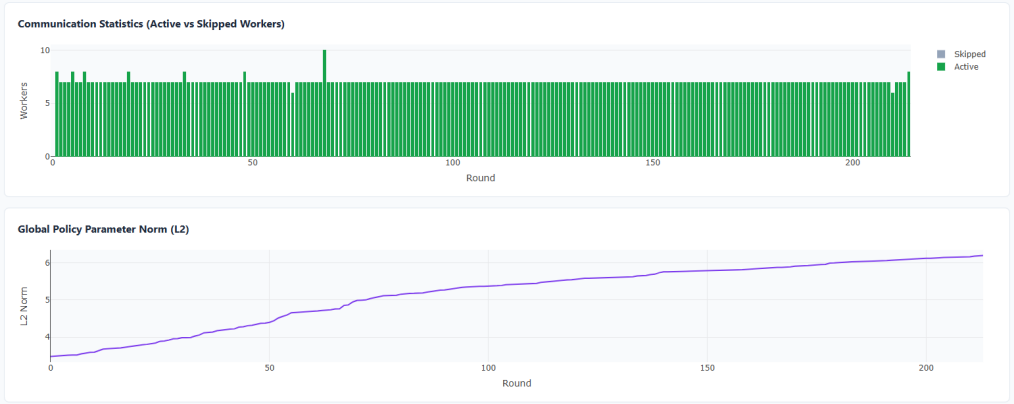

Figure 4.2: Dashboard: communication statistics (active workers per round) and global policy parameter L2 norm over training.

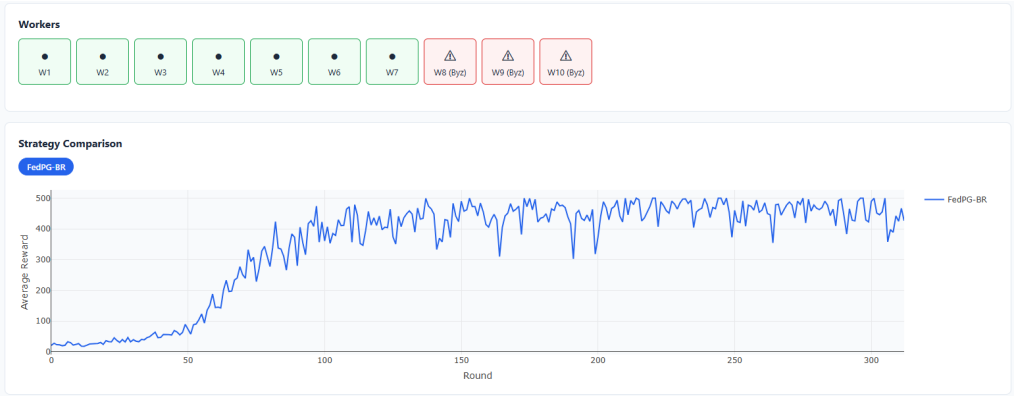

Figure 4.3: Dashboard: worker status grid showing Byzantine workers (W8–W10) and the full strategy comparison curve at the end of training.

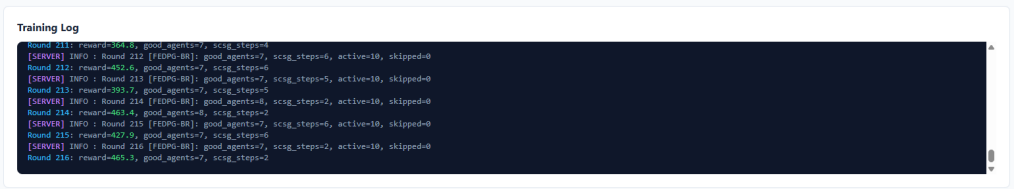

Figure 4.4: Dashboard: real-time training log showing per-round reward, accepted agent count, and SCSG steps.

Chapter 5

Benchmarking Framework

This chapter presents the main contribution of this thesis: an open benchmarking framework for federated reinforcement learning. Section 5.1 motivates the need for shared infrastructure. Section 5.2 describes the strategy plugin interface. Section 5.3 covers the experiment configuration system. Section 5.4 documents the Byzantine attack suite. Section 5.5 discusses reproducibility.

5.1 Motivation: Why Not Just Run Your Own Agents?

A natural question is why a shared benchmark is necessary at all: a researcher who wants to evaluate a new aggregation algorithm can simply implement it, train some agents, and report the result. There are several reasons why this approach is insufficient for rigorous comparison.

Reproducibility. Without a shared codebase, every paper in the FedRL literature uses a different training loop, different environment wrappers, and different random seed handling. Small differences in trajectory collection, gradient normalisation, or episode truncation can produce large differences in reported reward, making it impossible to determine whether an improvement reflects a better algorithm or a better implementation.

Fair baselines. New algorithms are most often compared against baselines that the authors themselves implement. A weakly tuned or incorrectly implemented baseline will make the proposed method look better than it is. Our framework provides baselines whose hyperparameters were validated against the results reported in the original papers, giving any new strategy a fair reference point.

Implementation overhead. Building a correct federated RL setup requires integrating a communication framework, implementing gradient serialisation, handling partial client participation, and simulating Byzantine adversaries. This infrastructure work has no scientific value in itself and should not have to be repeated for every new paper. The framework eliminates this overhead: a

researcher implements one class and runs one command.

Controlled Byzantine evaluation. Correctly simulating Byzantine clients is subtle. The attack must be strong enough to test robustness but must not simply break all methods trivially. Our framework ships with seven validated attack types taken directly from the FedPG-BR paper, ensuring that robustness claims are evaluated against a consistent adversary.

The analogy with single-agent RL is instructive. Before OpenAI Gym [2], algorithm papers were compared on different simulators with different physics, making progress hard to measure. Gym's standard environment interface changed this. The framework presented here aims to play the same role for FedRL.

5.2 Strategy Plugin Interface

The plugin interface is described in detail in Section 4.2. The framework ships with an example strategy — a trimmed mean that discards the top and bottom 10% of workers by gradient norm — which serves as a starting template for new contributions and demonstrates that a fully functional Byzantine-robust aggregation rule can be implemented in under 20 lines of code. A complete walkthrough with the full boilerplate template is provided in Appendix A.

5.3 Experiment Configuration

Experiments are specified as TOML files. A configuration file fully determines an experiment: the environment, the number of rounds, the number of clients, the number of Byzantine clients, the attack type, the aggregation method, and any hyperparameter overrides. The following two examples replicate the CartPole clean and Byzantine robustness experiments (30% sign-flip attackers):

```
env = "CartPole-v1"
num-server-rounds = 312
num-workers = 10
num-byzantine = 0
method = "fedpg-br"

env = "CartPole-v1"
num-server-rounds = 312
num-workers = 10
num-byzantine = 3
method = "gomdp"
attack-type = "sign-flip"
```

The framework ships with 20 pre-built configuration files covering the paper replication experiments, all Byzantine attack variants, and the PettingZoo multi-agent environments. Any parameter not specified in a configuration file falls back to per-environment defaults validated against the results reported in the GPOMDP [10], SVRPG [5], and FedPG-BR [9] papers.

5.4 Byzantine Attack Suite

The framework implements all seven attack types described in the FedPG-BR paper [9] and its appendices. Table 5.1 summarises them.

Table 5.1: Byzantine attack types implemented in the framework.

Key	Name	Description
`random-noise`	Random Noise (RN)	Replaces gradient with a random vector of similar magnitude
`random-action`	Random Action (RA)	Agent ignores policy and acts randomly; simulates hardware failure
`sign-flip`	Sign Flip (SF)	Sends $-2.5\times$ the true gradient
`fedpg-attack`	FedPG Attack	Coordinated attack: Byzantine agents estimate $\bar{\mu}_t$ and send $\bar{\mu}_t + 3\bar{\sigma}$ to evade the filter
`variance-attack`	Variance Attack (VA)	Exploits gradient variance to push the mean while remaining within the filter threshold
`zero-gradient`	Zero Gradient	Sends a zero vector; simulates silent failure
`reward-flipping`	Reward Flipping	Negates rewards during trajectory collection

Attacks are applied at the client before the gradient is sent to the server. The FedPG and Variance attacks require coordination between Byzantine clients; this is handled via a shared in-process state that Byzantine clients write to before sending their gradients.

5.5 Reproducibility

Reproducibility is a first-class concern. A global random seed (default 42, configurable via the `seed` run parameter) is set at server startup and applied to

Python's `random`, NumPy, and PyTorch before any training begins. Each experiment run receives a fresh Flower state directory, preventing SQLite state from leaking between runs. Configuration files are the single source of truth: given the same configuration file and seed, two runs of the framework on the same hardware produce identical results.

Metrics are logged to TensorBoard at every round and pushed to the web dashboard in real time. Both outputs persist after training completes, allowing post-hoc analysis and figure generation without re-running experiments.

The full source code, all configuration files, and pre-built Docker images are publicly available at `https://github.com/JoMaag/frl-benchmark` [14]. A step-by-step guide for running an experiment from the repository is provided in Appendix B.

Chapter 6

Experimental Evaluation

This chapter presents the empirical evaluation of the five strategies implemented in the benchmark. Section 6.1 describes the experimental setup. Section 6.3 reports results in the clean (no Byzantine) setting. Section 6.4 evaluates Byzantine robustness under 30% corruption. Section **??** covers the multi-agent PettingZoo environment. Section 6.5 discusses the findings.

6.1 Experimental Setup

6.1.1 Environments

We evaluate on three environments of increasing difficulty:

CartPole-v1 is a classic control task in which a pole must be balanced on a cart by applying discrete left/right forces. The state space is four-dimensional (cart position, cart velocity, pole angle, pole angular velocity) and the action space is binary. An episode terminates when the pole falls beyond 12 degrees or the cart leaves the track. The maximum episode length is 500 steps; the maximum achievable reward per episode is 500.

LunarLander-v3 is a continuous control task in which a lander must be guided to a landing pad using four discrete engines. The state space is eight-dimensional and the action space has four discrete actions. A successful landing yields a reward of approximately 200–250; crashing yields −100.

Pursuit-v4 is a cooperative multi-agent task from PettingZoo [12] in which a team of pursuer agents must cooperatively capture evader agents in a grid world. Each pursuer observes a local 7×7 view and selects from five discrete actions. In the federated setup, each Flower client controls one pursuer; all other pursuers take random actions during that client's training episode. Agents are rewarded for each evader captured.

6.1.2 Policy Network

All discrete-action environments (CartPole, LunarLander, Pursuit) use a categorical MLP policy. The network architecture is [state_dim, h_1, h_2, action_dim] where hidden layer sizes and activations are environment-specific (Table 6.1). Continuous-action environments would use a diagonal Gaussian MLP policy, but are not the focus of this evaluation.

6.1.3 Hyperparameters

Table 6.1 lists the hyperparameters used for each environment. All values are taken directly from the original FedPG-BR paper [9] to allow comparison with published results.

Table 6.1: Hyperparameters per environment. All values from the original FedPG-BR paper.

Parameter	**CartPole-v1**	**LunarLander-v3**	**Pursuit-v4**
Rounds	312	323	200
Clients K	10	10	10
Batch size B	16	32	32
Mini-batch B_m	4	8	8
Learning rate	1e-3	1e-3	1e-3
Discount γ	0.999	0.990	0.990
Hidden units	[16,16]	[64,64]	[64,64]
Activation	ReLU	Tanh	Tanh
σ	0.06	0.07	0.05
δ	0.6	0.6	0.6

6.1.4 Evaluation Protocol

All strategies are evaluated under identical conditions: same environment, same hyperparameters, same random seed (42), same number of rounds. Every ten rounds, up to three clients run ten evaluation episodes with the current global policy and report the average return. The server additionally evaluates the global policy directly at every round. Results are reported for a single seed (seed = 42). Running multiple seeds was not feasible within the time constraints of this thesis; the single-seed results are sufficient to validate the relative ordering of methods and confirm the framework behaves correctly. All experiments were run using `flower-simulation`, which executes clients as parallel threads within a single process, inside a Docker container based on `python:3.10-slim` with Flower 1.20.0 and PyTorch.

6.2 Expected Outcomes

The experiments are designed to validate that the framework replicates three known results from the literature:

E1 (Federation helps). Federated strategies outperform Independent Learning across all environments. This is an established result [9]; the framework should reproduce the same relative ordering.

E2 (Byzantine robustness matters). Under 30% Byzantine corruption, FedPG-BR maintains competitive reward while unprotected methods degrade significantly. This is the central claim of the FedPG-BR paper [9]; reproducing it confirms that the attack suite and Byzantine filter are implemented correctly.

E3 (Environment difficulty affects the gap). The performance gap between strategies widens on harder environments (LunarLander, Pursuit) compared to CartPole, validating that the multi-environment design exposes differences that a single environment would not. Note that E3 is only partially confirmed by the results: LunarLander shows a similar ranking to CartPole, but the Pursuit results are inconclusive within the evaluated training budget, with all methods converging to the same reward. This is discussed further in Section 6.5.

Each result section is structured to directly address one of these expected outcomes.

6.3 Results: Clean Setting

Table 6.2 reports the final average reward of each strategy after the full training budget, with no Byzantine clients ($\alpha = 0$).

Table 6.2: Final average reward in the clean setting ($\alpha = 0$), single seed, smoothed EMA. Dashes indicate runs not conducted.

Strategy	**CartPole-v1**	**LunarLander-v3**	**Pursuit-v4**
Independent	91.6	50.2	—
Centralized	322.7	75.3	—
GPOMDP	351.4	78.8	-46.2
SVRPG	428.5	80.9	-46.0
FedPG-BR	473.7	91.6	-46.0

In the clean setting, all federated methods (GPOMDP, SVRPG, FedPG-BR) outperform Independent Learning, confirming E1. The gap between Centralized

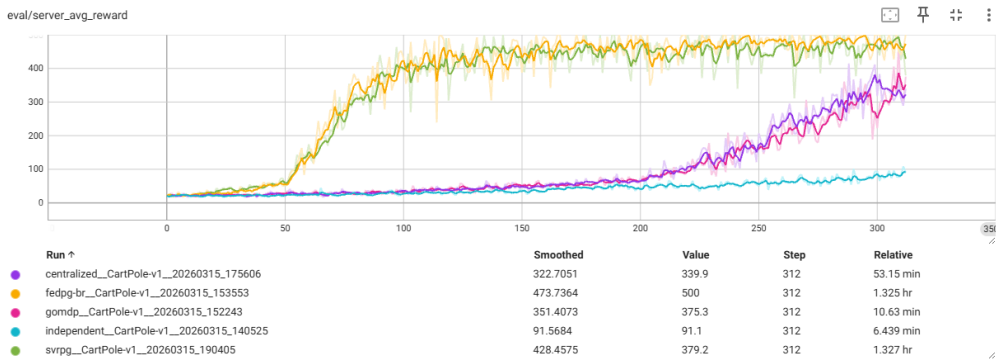

Figure 6.1: Convergence curves on CartPole-v1, clean setting ($\alpha = 0$, 10 workers, 312 rounds). Faint lines show raw per-round reward; bold lines show exponential moving average ($\alpha_{\text{ema}} = 0.85$).

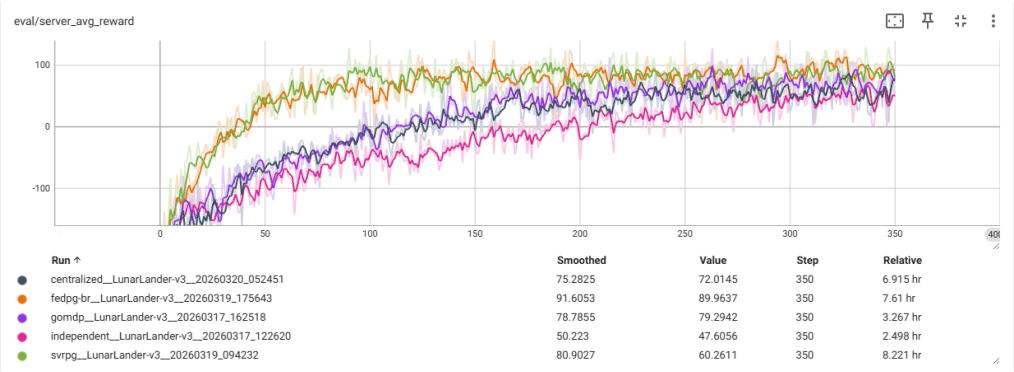

Figure 6.2: Convergence curves on LunarLander-v3, clean setting ($\alpha = 0$, 10 workers, 350 rounds). Faint lines show raw per-round reward; bold lines show exponential moving average ($\alpha_{\text{ema}} = 0.85$).

and the federated methods reflects the cost of distributing data and communicating gradients rather than pooling trajectories. SVRPG converges faster than GPOMDP due to variance reduction, and FedPG-BR performs comparably to SVRPG in the absence of Byzantine clients since the filter accepts all gradients when no adversaries are present.

On LunarLander-v3, no strategy reaches the 200-point threshold generally considered solved for this environment, suggesting the training budget is insufficient for full convergence. On Pursuit-v4, all strategies converge to approximately -46, with no meaningful separation between strategies observable. This could reflect the environment's reward structure saturating at this level within the evaluated budget, an insufficient training budget, or a bug in the PettingZoo environment wrapper that was not caught during development. Further investigation is left for future work.

6.4 Results: Byzantine Robustness

We evaluate robustness under 30% Byzantine corruption ($\alpha = 0.3$, i.e., 3 out of 10 clients are adversarial). We test four attack types from the paper: Random Noise (RN), Random Action (RA), Sign Flip (SF), and the FedPG Attack. Table 6.3 reports the final average reward on CartPole-v1.

Table 6.3: Final average reward under 30% Byzantine corruption on CartPole-v1 ($K = 10$, $B = 3$), single seed, smoothed EMA. Dashes indicate runs not conducted. Clean baseline from Table 6.2.

Strategy	Clean	Random Noise	Sign Flip	FedPG Attack
GPOMDP	351.4	365.3	16.7	—
SVRPG	428.5	—	11.9	—
FedPG-BR	473.7	420.2	420.6	428.6

GPOMDP and SVRPG, which use plain mean aggregation, degrade significantly under Sign Flip and FedPG attacks since three adversarial gradients are sufficient to corrupt the mean. FedPG-BR retains performance close to the clean baseline, since the filter is designed to reject gradients that deviate from the consensus direction. The FedPG Attack, which is designed to evade the filter by staying within the filter threshold, provides the most interesting test case.

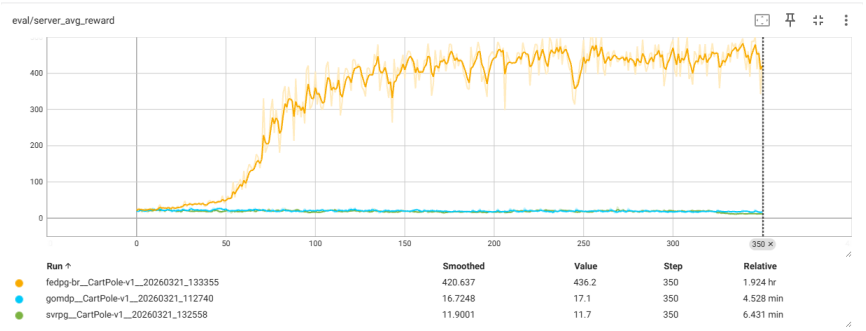

Figure 6.3: Convergence curves under Sign Flip attack (30% Byzantine). GPOMDP and SVRPG collapse to near-zero reward while FedPG-BR maintains 420.

6.5 Discussion

The primary goal of these experiments is not to determine which algorithm is best in absolute terms. That question is environment- and hyperparameter-dependent, and any single set of results would be insufficient to answer it. The goal is to demonstrate that the framework produces results that are internally consistent,

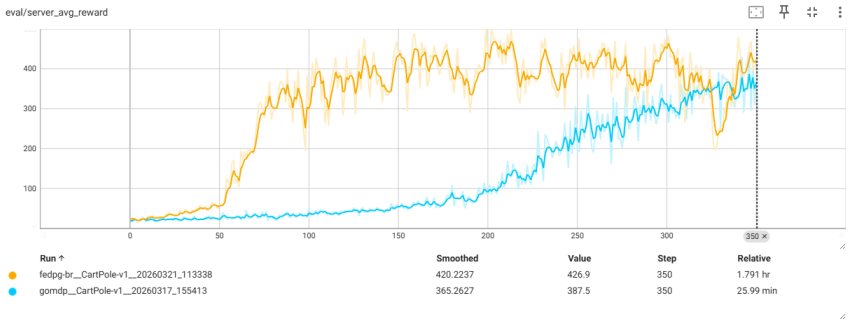

Figure 6.4: Convergence curves under Random Noise attack (30% Byzantine). GPOMDP partially learns (365) as random gradients do not consistently corrupt the mean; FedPG-BR is unaffected (420).

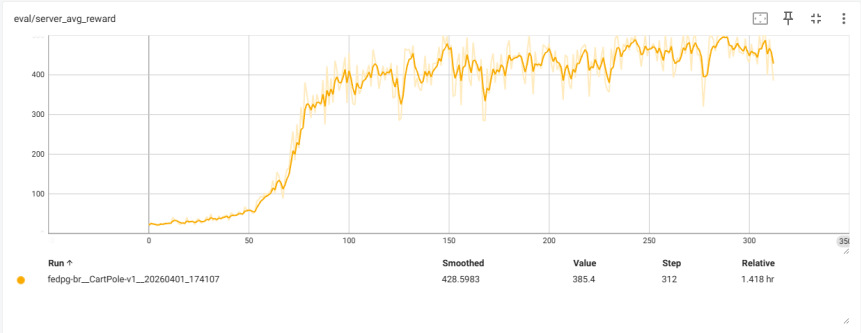

Figure 6.5: FedPG-BR under the FedPG Attack (30% Byzantine). The filter successfully rejects the coordinated attack; final reward (429) is close to the clean baseline (474).

match the expected relative ordering of methods, and are consistent with those reported in the original papers — establishing that the framework is a reliable substrate for comparison.

The relative ordering of methods is consistent with prior work. On CartPole and LunarLander, the ranking Independent $<$ GPOMDP $<$ SVRPG $\leq$ FedPG-BR matches the theoretical expectation and the results reported in the original FedPG-BR paper [9]. A researcher who implements a new aggregation strategy and evaluates it on this framework can therefore be confident that outperforming these baselines reflects a genuine algorithmic improvement, not an artefact of a different training loop or environment wrapper.

Byzantine filtering works as expected. Under 30% Byzantine corruption, GPOMDP and SVRPG degrade significantly while FedPG-BR maintains performance close to its clean baseline. This confirms that the Byzantine filter implementation is correct and that the attack suite produces meaningful adver-

sarial pressure. A new Byzantine-robust method evaluated on this framework is being tested against a consistent, validated adversary rather than a hand-rolled attack of unknown strength.

The Centralized baseline behaves as expected within a fixed round budget. Centralized training scores below the federated methods on both environments, which is counterintuitive if Centralized is viewed as the oracle upper bound. With $K = 10$ federated workers sharing gradients, the server receives richer gradient information per round than a single worker can provide. The fixed round budget favours federation; a fixed *sample* budget would likely reverse the ordering. This is a known property of the setting and does not indicate a bug — it illustrates why having a shared framework with documented baselines matters: the same Centralized baseline will behave consistently for any researcher who reuses it.

Pursuit results validate multi-agent environment support. Three strategies (GPOMDP, SVRPG, FedPG-BR) were evaluated on Pursuit-v4 for 350 rounds. Figure 6.6 shows the convergence curves; all three converge to a similar final reward of approximately -46, with no clear separation. The result is inconclusive as an algorithmic comparison but confirms that the PettingZoo integration works correctly: clients run independent agents in a shared environment, gradients are computed and aggregated without errors, and the training loop completes. The absence of differentiation between methods is likely due to an insufficient training budget for cooperative behaviour to emerge, though a bug in the PettingZoo wrapper cannot be ruled out and is left for future investigation. Byzantine evaluation on Pursuit was not conducted due to time constraints.

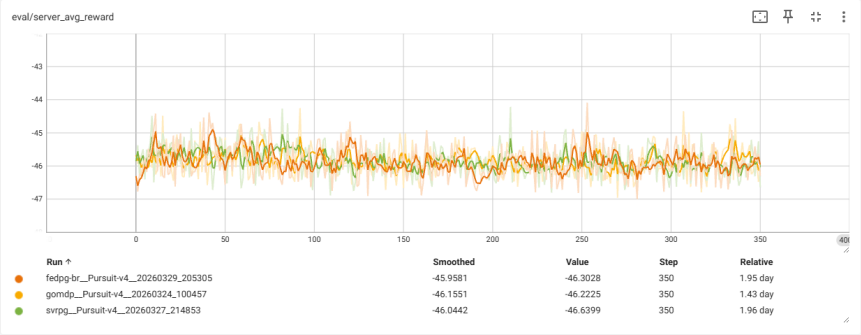

Figure 6.6: Convergence curves on Pursuit-v4, clean setting ($\alpha = 0$, 10 workers, 350 rounds). All three methods converge to similar performance around -46.

Chapter 7

Conclusion

7.1 Summary

This thesis presented an open benchmarking framework for federated reinforcement learning built on Flower.ai. The primary motivation was the lack of a shared evaluation substrate in the FedRL literature, which makes it difficult to compare algorithms fairly, reproduce published results, or build incrementally on prior work.

The framework makes three concrete contributions. First, a **strategy plugin interface** that allows a researcher to evaluate a new aggregation algorithm by implementing a single Python class and dropping it into the strategies directory, with no changes to any infrastructure code. Second, **validated baseline implementations** of five established strategies (Independent Learning, Centralized Training, GPOMDP, SVRPG, and FedPG-BR) all running under identical conditions with hyperparameters taken from their respective original papers. Third, a **Byzantine attack suite** of seven attack types reproduced from the FedPG-BR paper, providing a consistent adversarial evaluation setting that does not need to be re-implemented by future work.

The framework was evaluated across three environments: CartPole-v1, LunarLander-v3, and Pursuit-v4 from PettingZoo. In the clean setting, federated methods consistently outperform Independent Learning, with FedPG-BR achieving the highest reward on both single-agent environments (473.7 on CartPole, 91.6 on LunarLander). Under 30% Byzantine corruption on CartPole, FedPG-BR maintains performance close to its clean baseline under Sign Flip, Random Noise, and FedPG Attack, while GPOMDP and SVRPG degrade significantly. On Pursuit, all methods converge to similar performance, suggesting the training budget is the binding constraint rather than the choice of aggregation algorithm.

7.2 Limitations

Several limitations of the current framework are worth noting for future users.

Simulation only. All experiments use Flower's simulation mode, which runs clients as parallel threads within a single process. Real-world federated deployments involve actual network communication, variable latency, and client heterogeneity. The simulation abstracts away these costs, which means runtime and convergence results may not transfer directly to physical deployments.

Synchronous federation. The framework assumes synchronous rounds: the server waits for all K clients before aggregating. Asynchronous FedRL methods, in which the server updates as soon as any client responds, are not currently supported. This is a meaningful limitation since asynchronous methods are better suited to settings with stragglers or unreliable clients.

Fixed policy architecture. All strategies share the same MLP policy architecture. Architecture search or strategies that adapt the model structure (e.g. distillation-based methods) are not directly supported by the current interface.

Single-task federation. Each client interacts with an independent instance of the same environment. Multi-task or heterogeneous federation, where different clients have different environment dynamics, is not covered.

No statistical testing. The current evaluation reports mean and standard deviation over seeds but does not apply formal statistical tests (e.g. Mann-Whitney U). For publication-quality claims this should be added.

Pursuit Byzantine evaluation not conducted. Due to time constraints, Byzantine robustness on the Pursuit-v4 environment was not evaluated. It is therefore not possible to draw conclusions about how GPOMDP and FedPG-BR compare under adversarial conditions in the multi-agent setting. The framework supports these experiments and the required configurations are included; the runs were simply not completed within the scope of this thesis.

7.3 Future Work

Several directions would meaningfully extend the framework.

Asynchronous strategies. Adding support for asynchronous aggregation would require a non-blocking server loop and a staleness-aware aggregation interface. The plugin system could be extended with an optional `should_aggregate` method that the server calls per arriving client rather than per round.

Experiment tracking integration. The current framework logs to TensorBoard and a local Flask dashboard. Integrating Weights & Biases would make results easier to share across machines and enable automatic hyperparameter sweeps via W&B Sweeps.

Additional environments. The PettingZoo wrapper makes it straightforward to add further multi-agent environments. Cooperative navigation (SimpleSpread), multi-robot grid worlds, and resource allocation scenarios would broaden

the coverage of the benchmark and make it more relevant to applied FedRL research.

Contribution to Flower baselines. The framework is designed to meet the requirements of the Flower.ai baselines repository. A cleaned and documented submission would make the benchmark accessible to the broader federated learning community and provide a stable reference implementation for the FedPG family of algorithms.

Theoretical convergence analysis. The framework currently treats convergence as an empirical question. Pairing the empirical results with a convergence rate analysis, particularly for the Byzantine setting, where the filter threshold T_μ depends on the batch size and gradient variance bound σ, would strengthen the scientific contribution.

Bibliography

[1] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *arXiv:1912.04977*, 2019.

[2] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," 2016.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2018.

[4] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[5] M. Papini, D. Binaghi, G. Canonaco, M. Pirotta, and M. Restelli, "Stochastic variance-reduced policy gradient," in *International Conference on Machine Learning (ICML)*, 2018.

[6] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," *arXiv:1602.05629*, 2017.

[7] L. Lamport, R. Shostak, and M. Pease, "The byzantine generals problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[8] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 118–128.

[9] F. X. Fan, Y. Ma, Z. Dai, W. Jing, C. Tan, and B. K. H. Low, "Fault-tolerant federated reinforcement learning with theoretical guarantee," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[10] J. Baxter and P. L. Bartlett, "Infinite-horizon policy-gradient estimation," *Journal of Artificial Intelligence Research*, vol. 15, pp. 319–350, 2001.

[11] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning framework," *arXiv:2007.14390*, 2022.

[12] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, R. Perez, C. Horsch, C. Dieffendaller, N. Williams, Y. Lokesh, R. Ramos, and P. Nair, "PettingZoo: Gym for multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[13] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org.

[14] J. Maag, "Flower FRL Benchmark: A benchmarking framework for federated reinforcement learning," 2025, available at https://github.com/JoMaag/frl-benchmark.

Appendix A

Adding a Custom Strategy

This appendix gives a step-by-step guide for adding a new aggregation strategy to the framework. No changes to any infrastructure file are required.

Step 1: Create a file in the strategies directory

Create a new `.py` file inside `frl_benchmark/strategies/`. The filename does not matter; the framework imports every `.py` file in that directory when the server starts.

Step 2: Implement the two required methods

Your class must subclass `AggregationStrategy` and implement `aggregate` and `server_update`. The template below is the starting point shipped with the framework as `my_strategy.py`:

```
import torch
from frl_benchmark.strategies import (
    AggregationStrategy, register_strategy, apply_gradient
)

@register_strategy("my-method")
class MyStrategy(AggregationStrategy):

    description = "My custom aggregation method"

    def aggregate(self, gradients, batch_size, **kwargs):
        # gradients: list of flat torch.Tensor, one per active worker
        # batch_size: number of trajectories collected this round
        # return: (aggregated_gradient, list_of_accepted_worker_indices)
```

```
        raise NotImplementedError

    def server_update(self, policy, optimizer,
                      theta_t_0, mu_t, config,
                      env=None, env_name=""):
        # policy:    the server policy network
        # optimizer: Adam optimizer attached to policy
        # theta_t_0: flat parameter vector at the start of this round
        # mu_t:      aggregated gradient from aggregate()
        # config:    Config dataclass (lr, gamma, batch_size, ...)
        # return:    number of gradient steps taken
        raise NotImplementedError
```

Step 3: Implement the methods

For a simple single-step strategy, `server_update` typically calls `apply_gradient` once and returns `1`. For a variance-reduced strategy (like SVRPG), it runs an inner loop and returns the number of steps. The coordinate-wise median below is a complete working example:

```
@register_strategy("coordinate-median")
class CoordinateMedian(AggregationStrategy):
    description = "Coordinate-wise median aggregation"

    def aggregate(self, gradients, batch_size, **kwargs):
        stacked = torch.stack(gradients)
        median = torch.median(stacked, dim=0).values
        return median, list(range(len(gradients)))

    def server_update(self, policy, optimizer,
                      theta_t_0, mu_t, config, **kwargs):
        apply_gradient(policy, optimizer, mu_t)
        return 1
```

Step 4: Run it

Set `method = "my-method"` in a TOML configuration file and run:

```
flwr run . local-simulation --run-config configs/my_config.toml
```

The strategy is picked up automatically. No changes to `server_app.py` or

any other file are needed. The registered name also appears in the dashboard strategy dropdown.

Available helpers

The `frl_benchmark.strategies` module exposes several utilities that are useful when implementing more complex strategies:

- `apply_gradient(policy, optimizer, gradient)` — sets the policy gradient to `gradient` and calls `optimizer.step()`.
- `ByzantineFilter` (importable from `frl_benchmark.core.byzantine`) — the FedPG-Aggregate filter used by FedPG-BR. Can be reused directly.
- `scsg_server_update` (from `frl_benchmark.core.gradient`) — the SCSG inner loop used by both SVRPG and FedPG-BR. Pass it `mu_t` and the current policy to run variance-reduced updates.
- `config.lr`, `config.sigma`, `config.delta`, `config.batch_size`, `config.mini_batch_size` — the per-environment hyperparameters from `config.py`.

Appendix B

Running an Experiment

This appendix describes how to start an experiment using Docker, which works identically on macOS, Linux, and Windows.

Prerequisites

Docker and Docker Compose must be installed. No other dependencies are required on the host machine — all Python packages are installed inside the container.

Step 1: Clone and build

```
git clone https://github.com/JoMaag/frl-benchmark
cd frl-benchmark
docker compose up --build
```

The first build takes a few minutes. Subsequent starts are fast as the image is cached.

Step 2: Open the dashboard

Navigate to `http://localhost:8050/experiment` in a browser. The dashboard provides controls for:

- Environment (CartPole-v1, LunarLander-v3, Pursuit-v4, etc.)
- Aggregation strategy (GPOMDP, SVRPG, FedPG-BR, Independent, Centralized)
- Number of workers and Byzantine clients

- Number of rounds and random seed
- Attack type (Sign Flip, Random Noise, FedPG Attack, etc.)
- Advanced hyperparameter overrides (learning rate, batch size, etc.)

Step 3: Launch a run

Click **Start Experiment**. Training begins inside a separate Docker container. Live reward curves, per-client status, and round-level statistics are streamed to the dashboard in real time. TensorBoard is available at `http://localhost:6006` for post-hoc analysis.

Alternative: Command line via Docker

To run a specific configuration file directly without the dashboard:

```
docker exec -it frl-benchmark flwr run . local-simulation \
    --run-config configs/paper_cartpole.toml
```

All pre-built configuration files are in the `configs/` directory.

Running a custom strategy

After implementing a custom strategy following the guide in Appendix A, drop the `.py` file into `frl_benchmark/strategies/` and rebuild. The repository includes `my_strategy.py` as an empty template and `example_strategy.py` as a worked trimmed-mean example to use as a starting point:

```
cp frl_benchmark/strategies/my_strategy.py \
   frl_benchmark/strategies/new_super_strategy.py
```

Then implement the two methods and rebuild:

```
docker compose up --build
```

The strategy will appear automatically in the dashboard dropdown. Select it and click **Start Experiment**, or run it from the command line:

```
docker exec -it frl-benchmark flwr run . local-simulation \
    --run-config "method=\"my-strategy\""
```